

Теоретические основы информатики

Разделы	Темы
Информация и данные	Основные понятия информатики Кодирование информации
Двоичная математика	Системы счисления Битовые операции
Модели и структуры данных	Типы данных Структуры данных
Хранение и обработка данных	Структурированные файлы Парсинг данных Регулярные выражения
Методы информатики	Алгоритмизация Сортировка

Беляков Андрей Юрьевич

к.т.н., доцент

gitverse.ru/band/TOI

Тема 11

Раздел 5. Методы информатики

Тема 10

Алгоритмизация

- Способы описания алгоритмов.
- Оценка сложности алгоритмов.

Тема 11

Сортировка

- Методы сортировки сложностью $O(n^2)$.
- Методы сортировки сложностью $O(n * \log n)$.

Тема 11

Сортировка

Вопрос 1:

Методы сортировки сложностью $O(n^2)$

Вопрос 2:

Методы сортировки сложностью $O(n * \log n)$

Вопрос 1

Методы сортировки сложностью $O(n^2)$



Что такое сортировка

процесс упорядочивания элементов коллекции
по некоторому критерию (ключу)
в заданном порядке (возрастание, убывание)

Зачем нужна сортировка коллекций

Для лучшего восприятия списков данных пользователями

Для оптимизации задачи поиска

- в упорядоченном массиве можно искать бинарным поиском ($O(\log n)$)

Для анализ данных

- поиск медианы и квартилей, удаление дубликатов

Для сжатия данных

- сортируем по частоте и кодируем по Хаффману

Для повышения скорости поиска в базах данных

- строится сбалансированное дерево
- все узлы которого это отсортированные участки индексов

Свойства алгоритмов сортировки

Некоторые

Асимптотическая сложность

- зависимость количества операций от количества элементов
- это важно для производительности

Устойчивость

- сохраняет ли алгоритм относительный порядок равных элементов
- это важно для выполнения сортировок по нескольким параметрам

Сортировка на месте

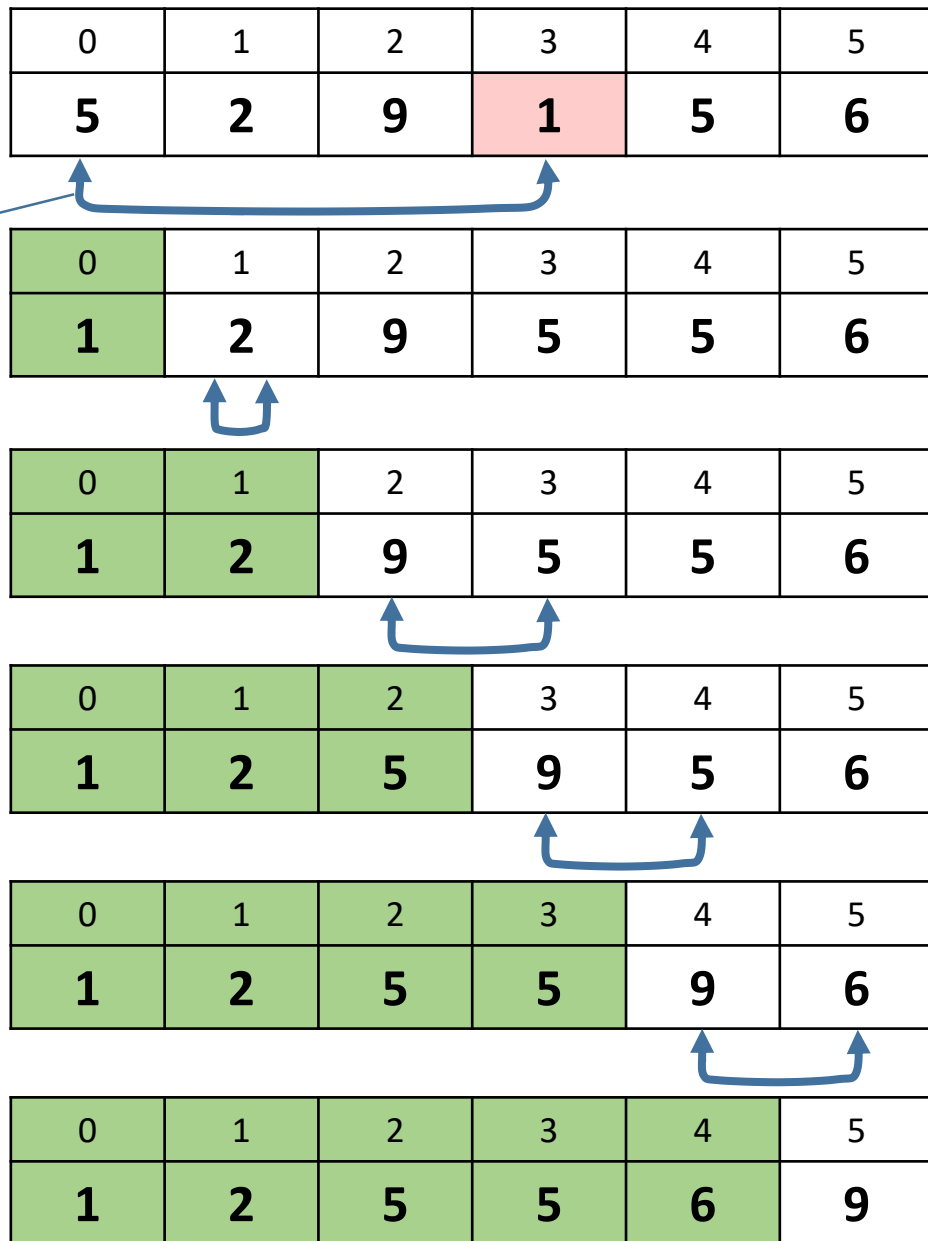
- требует ли алгоритм дополнительной памяти
- это важно при наличии ограничений или
- при очень больших коллекциях данных

Естественность

- работает ли алгоритм быстрее на почти отсортированных данных
- это важно при постоянном использовании коллекции

Сравнение алгоритмов

Алгоритм	Лучшее время	Среднее время	Худшее время	Память	Устойчивость	Примечание
Пузырьковая	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	+	С оптимизацией (флаг обмена)
Вставками	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	+	Лучшая для почти отсортированных
Выбором	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	-	Всегда квадратичная
Быстрая	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	-	Худший случай – неудачный pivot
Слиянием	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	+	Требует доп. Памяти
Пирамидальная (кучей)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	-	Гарантированное время



Сортировка выбором

неустойчивая

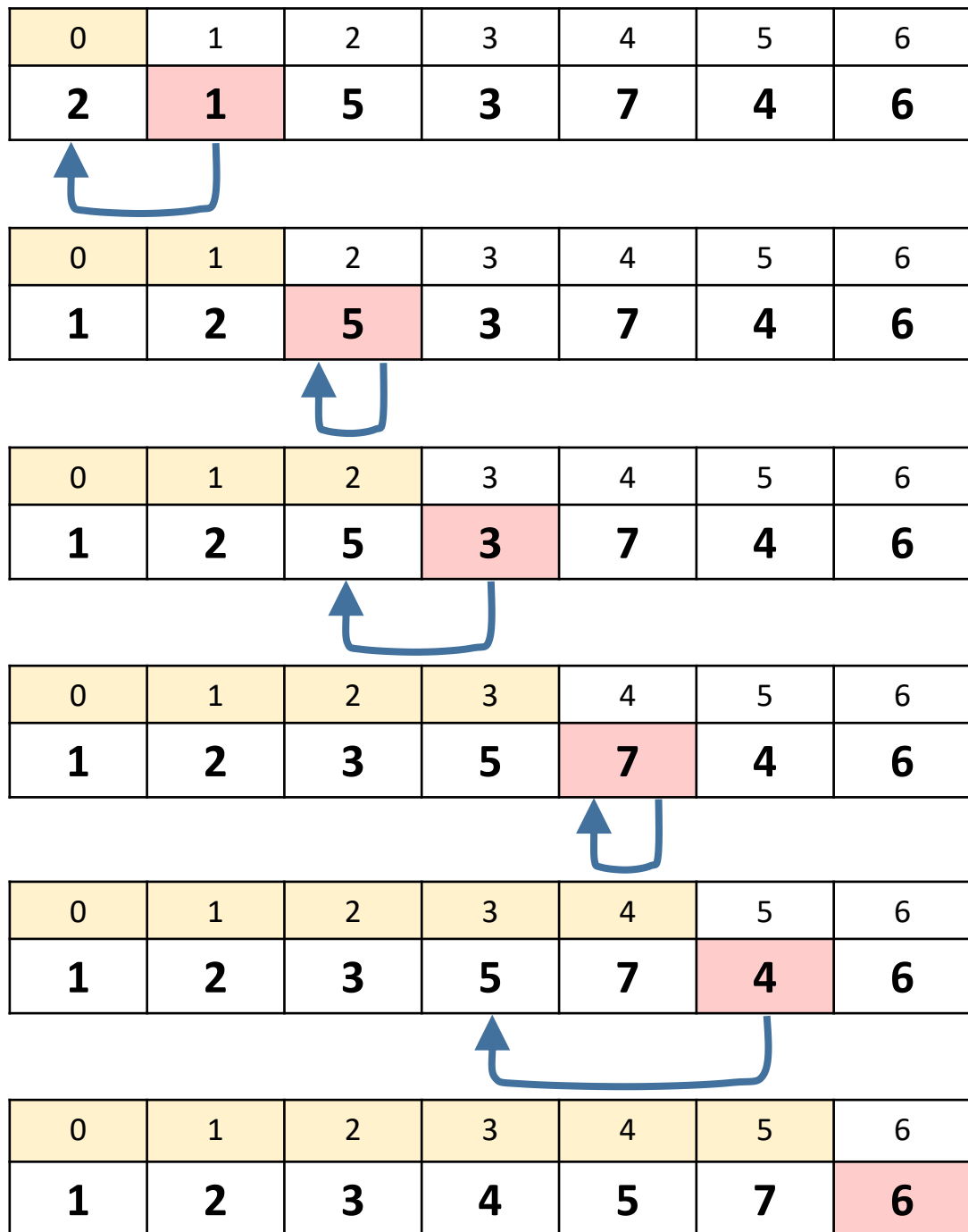
```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

всегда Внешний цикл двигает левую границу
уменьшая оставшуюся неотсортированную часть
всегда Внутренний цикл - ищет минимального в оставшейся части

$n * (n-1) / 2$ шагов $\Rightarrow O(n^2)$

Дополнительно только **1** ЯП

всегда n^2 шагов, но мало обменов



Сортировка вставками

естественная
устойчивая

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i] # подняли элемент
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key # вернули в список
    return arr
```

- в худшем случае каждый элемент нужно сдвинуть влево на длину его позиции и все элементы до него подвинуть вправо на 1 позицию
то есть $n * (n-1)/2$ шагов $\Rightarrow O(n^2)$

- в лучшем - массив отсортирован - каждый элемент сравнивается только 1 раз, всего $n-1$ сравнений
то есть $O(n)$

- в среднем - массив вперемешку - каждый элемент сравнивается примерно на половину пути
то есть $\sim n^2/4 \Rightarrow O(n^2)$

много обменов только в худшем случае

Дополнительно только 1 яп

всплытие первого пузырька

0	1	2	3	4	5
5	2	9	1	5	6
2	5	9	1	5	6
2	5	9	1	5	6
2	5	1	9	5	6
2	5	1	5	9	6
2	5	1	5	6	9

всплытие второго пузырька

0	1	2	3	4	5
2	5	1	5	6	9
2	5	1	5	6	9
2	1	5	5	6	9
2	1	5	5	6	9
2	1	5	5	6	9

всплытие третьего пузырька

0	1	2	3	4	5
2	1	5	5	6	9
1	2	5	5	6	9
1	2	5	5	6	9
1	2	5	5	6	9

если обменов не было, можно завершить сортировку

Сортировка пузырьковая

устойчивая

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

всегда

Внешний цикл двигает правую границу влево
уменьшая оставшуюся неотсортированную часть

всегда

Внутренний цикл - поднимает пузырёк до границы

$n * (n-1) / 2$ шагов $\Rightarrow O(n^2)$

Дополнительно только 1 ЯП

много проходов, если плохо сортирован
то много обменов

Контрольные вопросы

- для решения каких задач используются методы сортировки
- сравните алгоритмы сортировки по сложности
- что такое сортировка на месте
- устойчивость сортировки



Вопрос 2

Методы сортировки сложностью $O(n * \log n)$



- особенности реализации теории регулярок на практике
- на примерах реализации в Python
- какая библиотека и способы её использования
- какие есть методы
- как именно применять синтаксис регуляро

Литералы

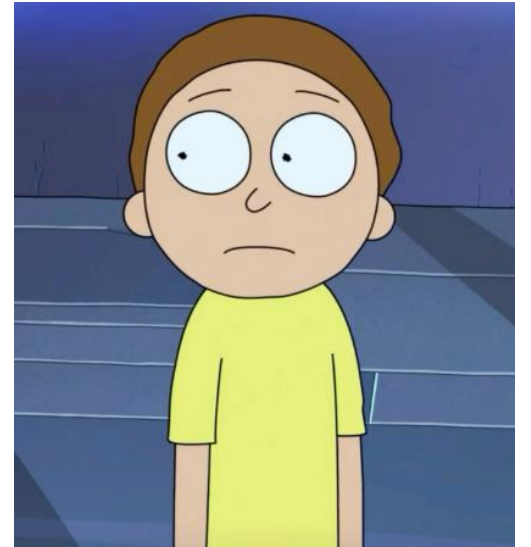
Литерал - это символ или последовательность символов, обозначают сами себя

Два способа использования регулярок:
метод скомпилированного паттерна или функция модуля `re`



Контрольные вопросы

- библиотека в Python для подключения функционала регулярок ?
- какие два способа **использования регулярок в Python ?**
- **основные методы/функции регулярок в Python ?**
- **флаги (модификаторы)**
- **как использовать символьные классы**
- **инвертированный символьный класс**
- особенности использования обратных ссылок



Теоретические основы информатики

Раздел 5. Методы информатики

Тема 10: Сортировка

2026

Беляков Андрей Юрьевич